

MODULE 2 · TOPIC 2

RTL Design Methodology

Basics of register transfer level (RTL) design · Overview of the RTL design process and methodology · Introduction to hardware description languages such as Verilog or VHDL

- Theory 1 · What RTL is — the two kinds of logic, the synchronous discipline, four levels of abstraction
- Theory 2 · The methodology — the flow, the synthesisable subset, the rules, and coding style
- Theory 3 · The patterns — datapath and controller, state machines, parameters and generate
- Theory 4 · HDLs — what one is, and Verilog against VHDL
- Practical · Labs A-N · 26 hours · 103 exercises · every number measured



Terminal Outcomes

Module 2 terminal outcomes

NOS NIE/ELE/N0102 · Verilog RTL Coding for Synthesis · 25 h theory + 35 h practical

1

Understand the design cycle of VLSI

Topics 1, 2

2

Understand Verilog syntax and LEVELS OF ABSTRACTION

Topics 2, 4, 5

3

Design and develop IPs for VLSI using Verilog

Topics 2, 4

4

Emulate, debug and characterise reusable IPs

Topics 2, 5, 6

Outcome 2 names this subtopic explicitly: "level of abstraction in Verilog programming".

Outcome 2 names this subtopic in the NOS itself

"Level of abstraction in Verilog programming" is the phrase. This session takes one circuit down all four levels, simulates them together, and proves they are the same — which is the deliverable the outcome asks for.



Key Learning Outcomes

Key learning outcomes for this subtopic

THEORY

- What register transfer level means, and the timing model it implies
- The levels of abstraction in an HDL, and how to choose between them
- How the synthesisable subset and the coding rules shape what you write
- Verilog against VHDL

PRACTICAL

- Write RTL for basic digital circuits
- Validate a design by simulation, with a testbench that checks itself
- Apply a coding standard, and check it mechanically rather than by eye
- Synthesise, and read what was built

Every one of them is assessed by a command you run.

Every outcome is assessed by a command you run, not by something you recite.



Which Command Assesses Which Outcome

Each outcome, and the command that assesses it

the outcome	the command
what RTL means	make transfer, make ladder
levels of abstraction	make ladder, make prove
coding style and what it costs	make mux, make pitfalls
the coding standard	make lint, make lintcheck
state machines and structure	make fsm, make dpctrl, make reuse
synthesis, and reading the result	make subset, make flow
Verilog or VHDL	make langs

Nothing in this topic is assessed by recitation.

Run the right-hand column and you have evidence for the left-hand one.



Every Syllabus Phrase, and Where It Is Covered

Every syllabus phrase, and where it is covered

syllabus phrase	section	slides
Basics of register transfer level (RTL) design	Theory 1	6-25
Overview of RTL design process and methodology	Theory 2	26-41
... and the patterns that process produces	Theory 3	42-59
Introduction to HDLs such as Verilog or VHDL	Theory 4	60-75
Practical: RTL Design and Implementation Labs	Labs A-N	76-92

The syllabus gives this subtopic 4 notional hours.

Enough to define the terms; not enough to make anyone believe them. Lecture from the summary slides to fit four hours; deliver it properly by running the lab alongside.

On the four notional hours

Four hours is enough to define the terms. It is not enough to make anyone believe them — and nothing in this subtopic is believed until it has been seen to happen. Lecture from the summary slides to fit four hours; run the lab alongside to teach it.



How This Session Runs

How this session runs

THEORY 1	What RTL is	the two kinds of logic, the discipline, abstraction
THEORY 2	The methodology	the flow, the subset, the rules, and coding style
THEORY 3	The patterns	datapath and controller, state machines, parameters
THEORY 4	HDLs	what an HDL is, and Verilog against VHDL
PRACTICAL	Labs A-N	26 hours, 103 exercises, all measured

RTL is not a language and it is not a tool. It is a way of thinking about hardware.

RTL is not a language and it is not a tool. It is a way of thinking about hardware.

THEORY 1

What RTL Is

The name is the definition, and almost everything else in this topic is a consequence of it.

- Registers, and what transfers into them on each clock edge
- Why `<=` and `=` are not interchangeable
- Four levels of abstraction, one circuit
- And what a synthesiser makes of each level



Register Transfer Level: the Name Is the Definition

Register Transfer Level: the name is the definition

1. WHICH REGISTERS EXIST

2. WHAT TRANSFERS INTO EACH,
ON EACH CLOCK EDGE



you never say how long this takes, or what gates it uses

```
always @(posedge clk) y <= x + 1;
```

x, through an adder, into y - and everything else is the tool's problem.

Two statements, and nothing else

1. Which registers exist. 2. What transfers into each one, on each clock edge.

How many gates, which gates, how they are wired, how fast they are — none of that is in the description. You state the transfers; synthesis works out the circuit.



Watch One Value Transfer

Watch one value transfer, one register per edge

A single 5 is applied on cycle 0 and never again.

cycle	din	x	y	z	acc
0	5	5	1	0	0
1	0	0	6	2	0
2	0	0	1	12	2
3	0	0	1	2	14
4	0	0	1	2	16

5 -> x

6 -> y

12 -> z

-> acc

One edge each. Nothing moved in between - that is the entire timing model of RTL.

A single 5 applied on cycle 0. It reaches x, then y as 6, then z as 12, then acc — one register per clock edge, and nothing moves in between.



Combinational and Sequential

The only two kinds of digital logic there are

Every RTL block you will ever write is some arrangement of these two.

COMBINATIONAL



output = $f(\text{inputs RIGHT NOW})$

no clock · no memory · no history

SEQUENTIAL



output = $f(\text{inputs AND the past})$

state changes only on the clock edge

There is no third kind.

There is no third kind. Every block in this course is an arrangement of these two.



Side By Side

Combinational and sequential, side by side

	COMBINATIONAL	SEQUENTIAL
what it is	logic gates	flip-flops, and the logic feeding them
how you write it	always @* ... =	always @(posedge clk) ... <=
what it remembers	nothing	one value per flip-flop
what goes wrong	an inferred latch, a glitch	a setup or a hold violation
what checks it	simulation, lint	simulation, lint, and STA

**The whole of RTL design is deciding what goes in the registers,
and what happens between them.**

Each row is a rule you will use every day. Learn which column you are in before you write a line.



The Same Distinction, In Code

COMBINATIONAL

always @(*) or assign

Blocking assignment. Every output written on every path, or the tool builds a latch you did not ask for.

No clock appears anywhere in the block.

Checked by: simulation, lint.

SEQUENTIAL

always @(posedge clk)

Non-blocking assignment. Reset appears here and only here.

One value remembered per flip-flop, updated on the edge and at no other time.

Checked by: simulation, lint, and static timing analysis.

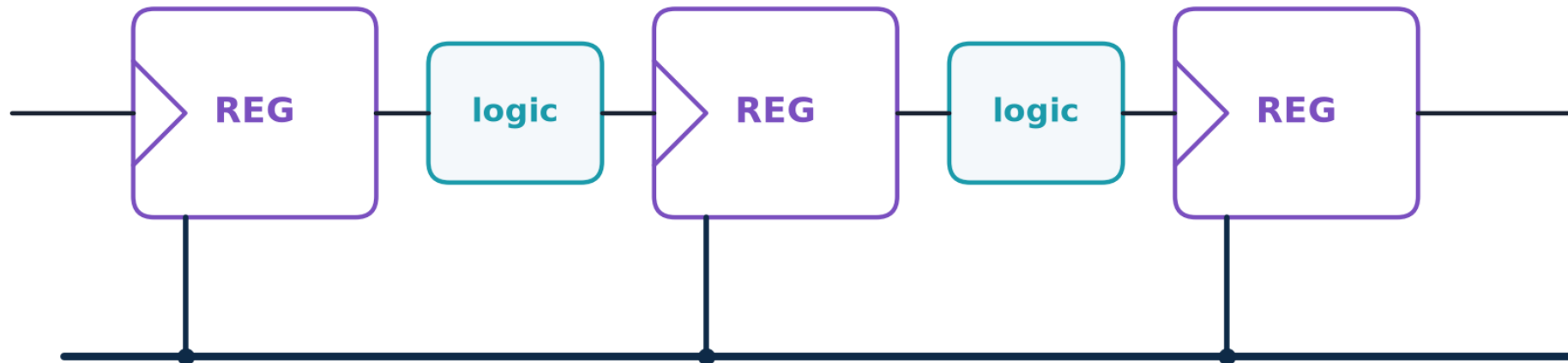
Why the two are kept in separate always blocks

A block that mixes them has to be read twice — once for what it computes and once for when. Splitting them means each block has one job, and the reviewer, the linter and the synthesiser all agree about which job it is.



One Clock, One Edge, Everything

The synchronous discipline: one clock, one edge, everything



clk one clock, reaching every register, and every register sampling at the same instant

You are allowed to break this. You are not allowed to break it by accident.

Almost every rule in this topic is a consequence of this one decision.



What the Discipline Buys You

ANALYSABLE

With one clock and one edge, timing analysis is a finite question: for every path from a flip-flop to a flip-flop, does the data arrive in time? Add a gated clock and the question multiplies.

COMPOSABLE

Two blocks written to the same discipline can be wired together without a conversation. Two blocks written to different disciplines need one, every time.

REVIEWABLE

A reviewer who knows the discipline can read your block for what it computes, because the question of WHEN has already been answered the same way it always is.

TESTABLE

Scan insertion, the technique that makes a chip testable after manufacture, assumes edge-triggered flip-flops on one clock. Latches and gated clocks each need special handling.

You are allowed to break these rules. You are not allowed to break them by accident.



Why Anyone Designs At This Level

	Behavioural / algorithmic	RTL	Gate netlist
you write	the algorithm	registers and transfers	every gate
timing	none at all	one clock period per stage	exact, per gate
synthesisable	rarely	yes — this is the target	yes, but why would you
a 10k-gate design	unbuildable	a few hundred lines	tens of thousands of lines
who writes it	architects, in C or SystemC	you	the synthesiser

RTL is the level where the trade lands correctly

High enough that a human can write and read a real design; low enough that a tool can build it without guessing at your intent.

Every industrial digital design in the last thirty years was written here. That is not fashion — it is where the abstraction pays for itself.

Above RTL you cannot say when things happen. Below it you cannot say anything else.



The Two Assignment Operators, and Why It Matters

Why clocked blocks use <= and combinational blocks use =

<= non-blocking

in CLOCKED blocks

every right-hand side is read first,
using the OLD values, then every
left-hand side updates at the
same instant

= blocking

in COMBINATIONAL blocks

each statement completes before
the next one starts, exactly the
way software statements run

a <= b; b <= a; swaps them.

a = b; b = a; does not.

Neither is an error. Both are caught by rules L001 and L002 of the linter in this lab.

Neither one is an error. Both are caught by the linter you build in this topic.



The Swap, Worked Through

```
// NON-BLOCKING - inside always @(posedge clk)
// Step 1: read every right-hand side, using the OLD values.
// Step 2: update every left-hand side, all at the same instant.
```

```
    a <= b;      // reads old b
    b <= a;      // reads old a
```

a and b are SWAPPED. This is what a hardware register does.

```
// BLOCKING - inside always @*
// Each statement finishes before the next one starts.
```

```
    a = b;       // a is now b
    b = a;       // a is already b, so this does nothing
```

BOTH end up holding b. This is what software does.

Why the rule is absolute rather than stylistic

Two clocked blocks using blocking assignments can see each other's half-updated values, and which one wins depends on the order the simulator happens to evaluate them in — an order the language standard deliberately does not fix.

Non-blocking assignment exists precisely to make that race impossible.



And What Each One Actually Builds

Blocking in a clocked block: what it actually builds

shift_nb — correct

```
always @(posedge clk) begin
    q[0] <= din;
    q[1] <= q[0];
    q[2] <= q[1];
end
```

3 flip-flops

shift_bl — not a shift register

```
always @(posedge clk) begin
    q[0] = din;
    q[1] = q[0];
    q[2] = q[1];
end
```

1 flip-flop

0 wrong cycles against 6. Nothing illegal was written, so nothing warned.

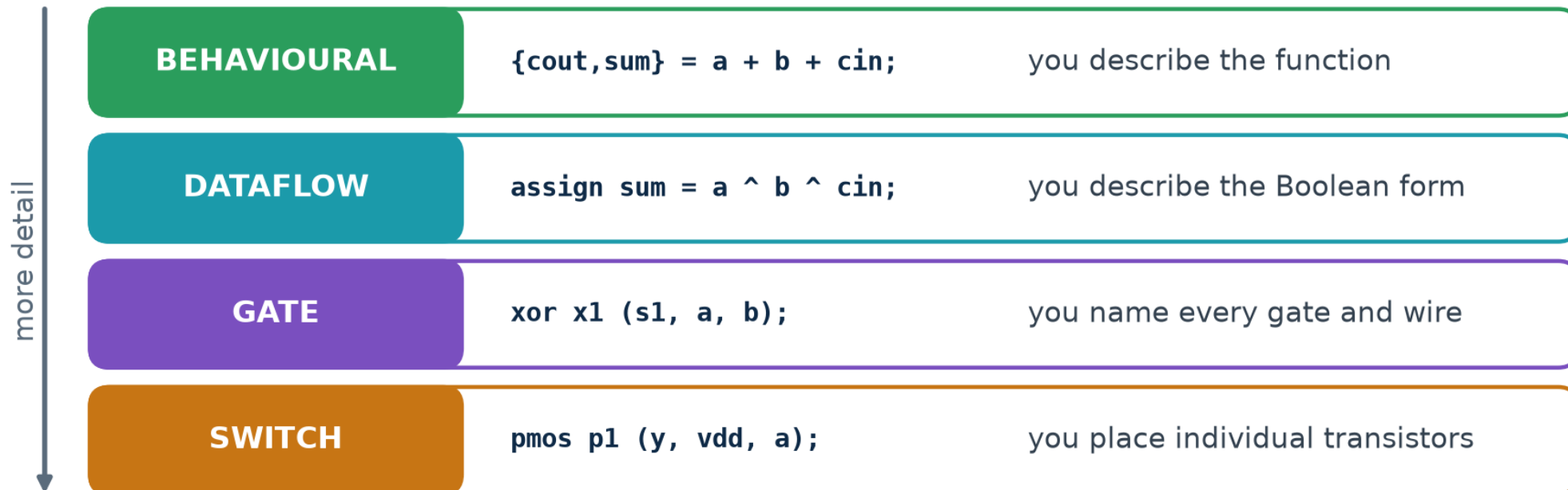
Three flip-flops against one. Nothing illegal was written, so nothing warned.



Four Levels, One Circuit

Four levels of abstraction, one circuit

Higher up you say WHAT. Lower down you say HOW. The circuit is the same.



All four simulated together on all 8 input patterns. Zero mismatches.

The level of abstraction changes WHAT WAS WRITTEN, not what it does.



The Same Adder, Written Four Ways

```
// BEHAVIOURAL - you describe the function
always @* {cout, sum} = a + b + cin;

// DATAFLOW - you describe the Boolean form
assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);

// GATE - you name every gate and every wire
xor x1 (s1, a, b);      xor x2 (sum, s1, cin);
and a1 (ab, a, b);      and a2 (bc, b, cin);  and a3 (ac, a, cin);
or  o1 (cout, ab, bc, ac);

// SWITCH - you place individual transistors
pmos p1 (y, vdd, a);    nmos n1 (y, gnd, a);    // one CMOS inverter
```



And What a Synthesiser Makes of Each

And what a synthesiser makes of each level

written at	synthesis	cells	what came out
behavioural	OK	5	NAND x3, XOR x2
dataflow	OK	6	AND x2, OR x2, XNOR, ORNOT
gate	OK	6	identical netlist to dataflow
switch	REFUSED	-	transistors are not synthesisable

The BEHAVIOURAL description produced the SMALLEST circuit - five cells against six.

Dataflow and gate produced the IDENTICAL netlist: writing the Boolean form already fixes the structure.

So write at the highest level that expresses your intent.

The behavioural description produced the SMALLEST circuit — and dataflow and gate produced the identical netlist.



The Rule This Gives You

Write at the highest level that expresses your intent

Every level you descend takes a decision away from the tool and gives it to you — whether or not you wanted it.

What the tool is better at than you

Choosing a gate mix for a Boolean function.

Balancing a logic tree.

Meeting a timing constraint by restructuring.

Doing all of that again, consistently, at 3 a.m.

What you are better at than the tool

How many clock cycles the job should take.

Where the registers go.

What is shared and what is duplicated.

What the interface should look like.

Give the tool the first list. Keep the second one for yourself.



Simulation Shows. Proof Settles.

Simulation shows. Proof settles.

EXHAUSTIVE SIMULATION

8 patterns, 3 inputs.
Every possible case, so this
really is complete.

but 2^{30} patterns is not

EQUIVALENCE CHECKING

Both designs, same inputs,
outputs compared, and an
assertion that they agree.

A solver cannot break it

behav vs dataflow EQUIVALENT (94 SAT variables)

behav vs broken NOT EQUIVALENT

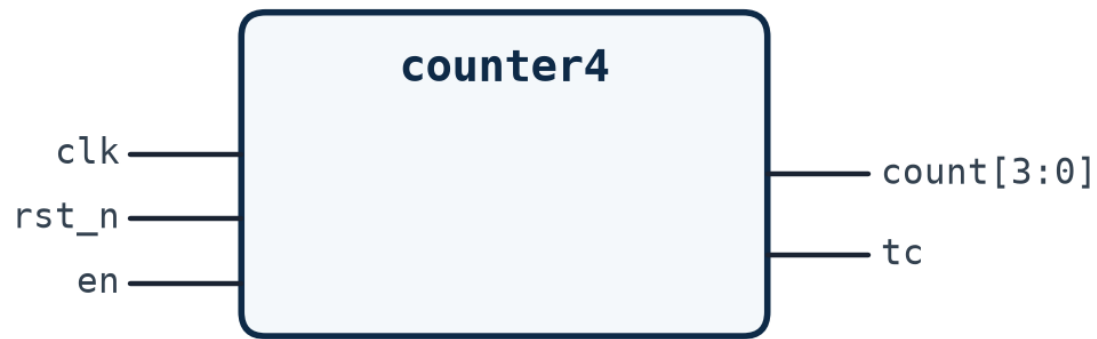
A checker that cannot fail is not evidence of anything.

A checker that cannot fail is not evidence of anything — which is why the lab includes a deliberately broken adder.



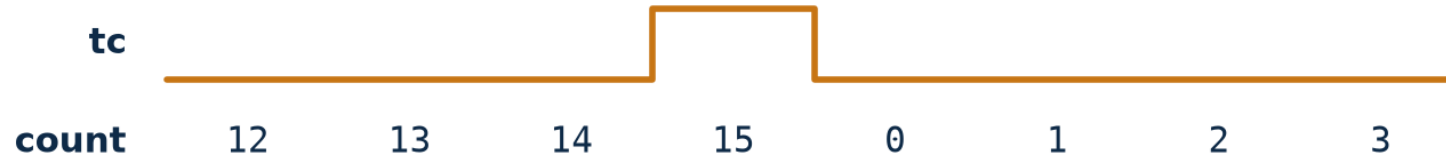
One Design, Carried All the Way Through

The running example: a 4-bit counter



```
always @(posedge clk or negedge rst_n)
    if (!rst_n) count <= 4'd0;
    else if (en) count <= count + 4'd1;

assign tc = en & (count == 4'd15);
```



tc is high for exactly one cycle, while count == 15

Four bits, an asynchronous reset, an enable and a terminal count. Small enough to read in one glance, big enough to be a real design.



Read It Line By Line

```
module counter4 (  
    input          clk,  
    input          rst_n,      // asynchronous, active LOW  
    input          en,  
    output reg [3:0] count,  
    output         tc          // one cycle high at 15  
);  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)      count <= 4'd0;      // reset wins  
        else if (en)      count <= count + 4'd1;  
    end                    // no else: HOLD  
  
    assign tc = en & (count == 4'd15);  
endmodule
```

Four decisions are visible in fourteen lines

The reset is ASYNCHRONOUS, so it is in the sensitivity list; it is ACTIVE LOW, so the port is named `rst_n` and the test is `!rst_n`. Reset is tested FIRST, so it wins over the enable. And the missing else is deliberate — in a CLOCKED block, no assignment means hold, which is what a flip-flop does anyway.



The Numbers, Worked Out

A worked numerical example

counter4 at 100 MHz

clock period	10 ns
states before it wraps	$2^4 = 16$
one full cycle	160 ns
tc pulse rate	6.25 MHz
tc pulse width	10 ns

accum_top, N = 6 samples

cycle 0	start seen, cnt <- 6
cycles 1..6	one sample added per clock
cycle 7	counter reached zero
cycle 8	done pulses, sum valid
total latency	$N + 3 = 9$ clocks

Derive the formula, then measure it. Measured: sum = 157, done at cycle 8.

Derive the formula, then measure it. A testbench that only ever tries one value of N has told you almost nothing.



Nine Questions

#	Question	The answer in one line
1	What two things does an RTL description state?	which registers exist, and what transfers into them
2	What does <code><=</code> mean inside a clocked block?	read all right-hand sides first, then update together
3	Name the four levels of abstraction.	behavioural, dataflow, gate, switch
4	Which level produced the smallest netlist, and why?	behavioural — it left the tool free to choose
5	Why did dataflow and gate give identical netlists?	writing the Boolean form already fixes the structure
6	Why prove equivalence instead of simulating?	exhaustive simulation stops being possible at ~30 inputs
7	What are the only two kinds of digital logic?	combinational and sequential — there is no third
8	Name three things the synchronous discipline buys you.	analysable timing, composable blocks, testable silicon
9	In counter4, why is there no else on the enable?	in a clocked block, no assignment means hold

Theory 2 asks what you are allowed to write, and why the rules exist.

THEORY 2

The RTL Design Process and Methodology

Methodology is the hardest thing to teach, because the honest version is a list of habits and the dishonest version is a list of slogans.

- The design flow, and where this topic sits in it
- The synthesisable subset — measured, not asserted
- The bug that makes simulation and silicon disagree
- Seven coding rules, and a tool that checks them
- Micro-architecture: the decisions RTL cannot make for you



The RTL Design Flow

The RTL design flow, and where this topic sits

SPECIFICATION	what it must do, in words, before any code	
MICRO-ARCHITECTURE	cycles, registers, partitioning	<i>Topic 2</i>
RTL CODING	the design, written in an HDL	<i>Topics 2, 4</i>
LINT	coding rules, checked mechanically	<i>Topic 2</i>
SIMULATION	does it do what the spec says?	<i>Topic 5</i>
SYNTHESIS	RTL becomes a gate netlist	<i>Topics 2, 4</i>
GATE-LEVEL CHECK	does the netlist still do it?	<i>Topic 2</i>
TIMING ANALYSIS	is it fast enough?	<i>Topic 6</i>
PLACE AND ROUTE	gates become geometry	<i>Module 3</i>
SIGN-OFF	every check passes at every corner	<i>Module 3</i>

Every arrow points both ways in practice: a timing failure sends you back to the RTL.

Every arrow points both ways in practice: a timing failure sends you back to the RTL, a synthesis surprise back to the micro-architecture.



Executed, Not Described

A methodology is a set of gates, not a set of suggestions

make flow - seven stages on one design, each producing evidence.

	stage	what runs	evidence produced
1	SPEC	written before the RTL	4 sentences
2	LINT	tools/rtl_lint.py	0 issues
3	RTL SIMULATION	iverilog, 18 cycles	wraps at 15
4	SYNTHESIS	yosys	12 cells
5	GATE SIMULATION	the netlist, same stimulus	18 cycles
6	COMPARE	RTL against gate transcript	identical
7	PROVE	formal equivalence	proven by induction

Stage 6 says they agree on the 18 cycles tested. Stage 7 proves they agree on EVERY input sequence.

make flow stops at the first stage that fails. That is what makes it a methodology rather than a diagram.



Which Verilog Actually Synthesises

Which Verilog actually synthesises - measured

construct	synth	cells	what the tool said
always @* with full case	OK	10	clean
clocked block with <=	OK	8	8 flip-flops
if with no else	OK	1	inferred a LATCH
incomplete sensitivity list	OK	1	built anyway
#5 delay in RTL	OK	1	the delay silently vanished
initial block	OK	10	accepted - FPGA only, not ASIC
for loop, constant bounds	OK	7	unrolled into 7 gates
while loop, data-dependent	REFUSED	-	not a constant function
real (floating point)	REFUSED	-	syntax error
a / b variable divisor	OK	371	a full divider
a / 4 constant divisor	OK	0	no logic at all - wires

371 cells against 0, for the same operator.

The difference is entirely what you divided BY.

Eleven constructs, run through a real synthesiser. The table is measured, not remembered.



Three Rows Worth Discussing

371 cells against 0

`a / b` where `b` is a signal builds a full combinational divider — 371 cells.

`a / 4` builds nothing at all. It is a rename of wires.

Same operator. The difference is entirely what you divided BY.

#5 vanished silently

A delay is a simulation instruction. Silicon has no way to wait five time units.

Synthesis ignored it and built the gate. Your simulation and your chip now behave differently.

initial was accepted

Yosys targets FPGAs, where the bitstream really does initialise the flops.

An ASIC flow would not. This is how code that works on an FPGA fails on an ASIC.

The habit this table is for

Before you write a construct you are unsure about, synthesise a ten-line module containing only that construct and read what came out. It takes two minutes and it is the only way to actually know.



The Most Common RTL Bug There Is

The inferred latch: the most common RTL bug there is

```
always @* begin
    if (en) y = d;
end
```

no else - so you never said what y is when en = 0



\$_DLATCH_P_

A level-sensitive latch you did not ask for - and it is not an error, so nothing stops you.

It is not an error. The tool builds it, mentions it in a log nobody reads, and hands you the design.



Why a Latch In the Middle of Your Logic Hurts

	A flip-flop	A latch
captures	on the clock EDGE	while the enable is HIGH
is transparent	never	for the whole enable window
timing analysis	one clean check per edge	time-borrowing; much harder to analyse
in a scan chain	standard	usually needs special handling
you asked for it	yes, by writing posedge	no — you forgot an else

So the rule is mechanical

In a combinational block, assign every output on every path. An if needs an else; a case needs a default. If you would rather not write them out, assign a default value at the top of the block and let later assignments override it.

Rules L005 and L006 of the linter catch this, and make lintcheck confirms the linter agrees with Yosys on every file.



When Simulation and Silicon Disagree

When simulation and silicon disagree

```
always @(a) begin  y = a & b;  end    - b is read, but not in the list.
```

stimulus	a	b	RTL y	NETLIST y	
start	0	0	0	0	
change a	1	0	0	0	
change b	1	1	0	1	DISAGREE
change b	1	0	0	0	
change a	0	0	0	0	
change b	0	1	0	0	

Your testbench was verifying a circuit that will never be built.

One disagreement in six - and nothing anywhere reported an error. Never maintain a sensitivity list by hand.

One disagreement in six — and nothing anywhere reported an error.



Why This Class of Bug Is Special

Most bugs are found by testing. This one cannot be, because **the thing you are testing is not the thing that will be built**. Every test you write passes, and the chip still fails.

Cause	What simulation does	What synthesis does
incomplete sensitivity list	the block sleeps; the output goes stale	ignores the list; builds the logic
# delays in RTL	waits the specified time	ignores them entirely
initial block (ASIC)	sets the value at time zero	ignores it; the flop powers up unknown
blocking in a clocked block	result depends on evaluation order	builds one specific circuit

All four have the same cure

Use always @* and never a hand-written sensitivity list. No delays in RTL. A real reset instead of an initial block. Non-blocking in clocked blocks. Then lint for all four, so that nobody has to remember.



Seven Rules, Checked By a Tool

Seven rules, checked by a tool instead of by memory

rule	what it catches	why it matters
L001	blocking (=) in a clocked block	two blocks see half-updated values
L002	non-blocking (<=) in a combinational block	simulates as a register, builds as logic
L003	= and <= mixed in one block	no reader can tell the intent
L004	explicit sensitivity list	you maintain it, for ever
L005	if with no else in a combinational block	infers a LATCH
L006	case with no default	infers a LATCH
L007	signal driven from two always blocks	two drivers

16 files · linter and Yosys agree on every one · 0 disagreements

L005 and L006 are not opinions - they predict what a synthesiser will build, and make lintcheck settles it.

A linter that cries wolf gets switched off. One that stays quiet is worse than none at all.



The Coding Standard, In One Place

The RTL coding rules, in one place

Combinational logic

always @* - never a hand-written sensitivity list

blocking assignments (=) only

assign every output on every path

Sequential logic

always @(posedge clk) - one clock, one edge

non-blocking assignments (<=) only

one signal, one always block, ever

Never in synthesisable RTL

delays, initial, real, unbounded loops

both clock edges in one path

a clock built from logic - use an enable

None of these are style preferences. Each one exists because breaking it produces a simulate/build mismatch.

None of these are style preferences. Every one exists because breaking it produces a design that simulates differently from the way it is built.



The Decisions RTL Cannot Make For You

Micro-architecture: the decisions RTL cannot make for you

the decision	the options	what it costs you
How many clock cycles?	one big cycle, or a pipeline	Fmax, latency, area
Where do the registers go?	stage boundaries	Fmax, directly
What is shared?	one multiplier reused, or four in parallel	area against throughput
How is it partitioned?	module boundaries, hierarchy	readability and reuse
What is the interface?	handshake, valid/ready, fixed latency	how easily it plugs in
Where does reset reach?	everything, or only what needs it	area, routing congestion

A synthesiser is better than you are at choosing gates.

It is not better than you at any row above, and it will not warn you that you chose badly.

Make these on paper, before the first line of RTL. A synthesiser will faithfully build whatever you chose.



Writing RTL That Someone Else Can Use

Writing RTL that someone else can use

Parameterise, do not duplicate

WIDTH as a parameter, not four copies

One clock, stated in the name

clk, and which domain it belongs to

Reset policy, written down

sync or async, high or low - pick one

No magic numbers

a named localparam beats 8'd47 in a case

An interface, not a pile of wires

valid/ready is understood everywhere

A testbench that ships with it

IP without one is a liability, not an asset

Could a colleague drop it into another design next year without asking you anything?

Module 2's fourth terminal outcome asks for this by name

"Emulate, debug and characterise reusable IPs." Reuse is not something you add to a design afterwards — it is a set of decisions you take while writing it, and every one of them is listed above.



One Function, Three Coding Styles

One function, three coding styles - and three netlists

A 4:1 multiplexer. All 64 patterns simulated; every pair proved equivalent by SAT.

mux4_assign

```
assign y =  
    sel[1] ? (sel[0]?d[3]:d[2])  
          : (sel[0]?d[1]:d[0]);
```

3 cells

mux4_if

```
if      (sel==2'b00) y = d[0];  
else if (sel==2'b01) y = d[1];  
else ...           else y = d[3];
```

6 cells

mux4_case

```
case (sel)  
    2'b00: y = d[0];    ...  
    default: y = 1'b0;
```

10 cells

EQUIVALENT is not the same as IDENTICAL.

Proved equivalent by SAT — and still three different netlists.



What To Take From That

EQUIVALENT is not IDENTICAL

A formal equivalence proof answers exactly one question: do these two descriptions compute the same function?

It says nothing about area, nothing about timing and nothing about power. Those are separate questions with separate tools.

MEASURE YOUR OWN TOOL

The claim 'the optimiser flattens the difference' is the most repeated statement in RTL teaching, and on this toolchain it is false: 3, 6 and 10 cells.

A different synthesiser may well close the gap. Find out rather than assume.

SO WHICH DO YOU WRITE?

Whichever reads best to the next person, which is usually the case statement for anything with more than three branches. Then measure, and only reach for the terse form if the block is actually on a critical path.

Why the three differ at all

The conditional expression hands sel[1] and sel[0] straight to the multiplexer selects. The case and if/else versions ask the tool to build equality comparators against 2'b00, 2'b01 and so on, and then to work out for itself that those comparisons ARE the select bits. It gets part of the way.



Nine Questions

#	Question	The answer in one line
1	Name the first four stages of the flow.	spec, micro-architecture, RTL coding, lint
2	Why lint before simulating?	it costs seconds and catches what tests cannot
3	What builds a latch?	a combinational block that does not assign on every path
4	a / b against a / 4 — what did they cost?	371 cells against 0
5	Why is an incomplete sensitivity list so dangerous?	simulation and synthesis then build different circuits
6	What does # do in synthesisable RTL?	nothing at all — it is silently ignored
7	Name three decisions synthesis will not make for you.	cycle count, register placement, sharing, interface
8	Does formal equivalence say anything about area?	no — equivalent is not the same as identical
9	The three mux styles: how many cells each?	3, 6 and 10 — style did reach the netlist here

Theory 3 is about the patterns every real block is built from.

THEORY 3

The Patterns Every Block Is Built From

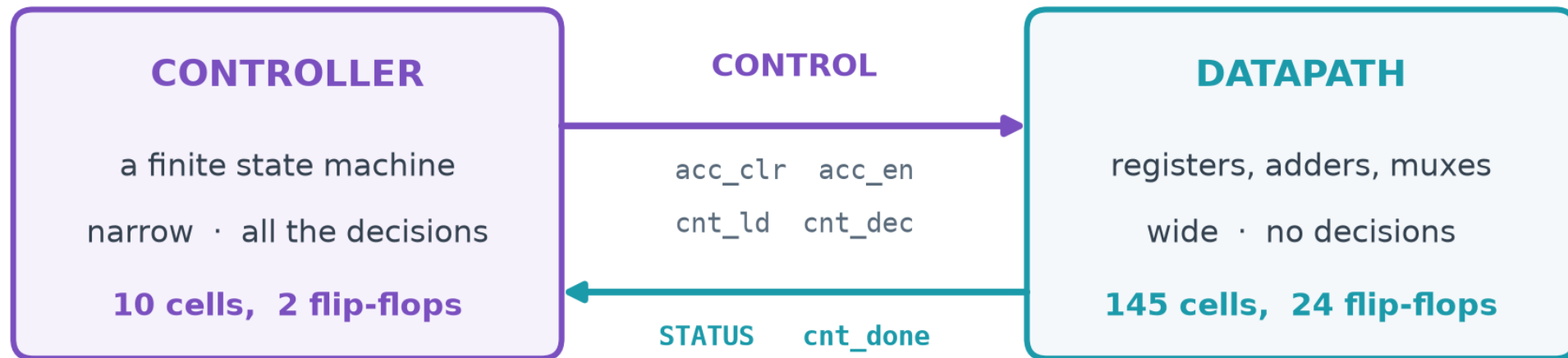
Three structures account for almost all synthesisable RTL. Once you can recognise them, most designs stop looking unfamiliar.

- Datapath and controller — the split that organises everything else
- The finite state machine, and the three-block way to write one
- Moore against Mealy, in structure and in measured timing
- State encoding, and what it actually costs
- Parameters, hierarchy and generate: from a module to an IP



Datapath and Controller

Datapath and controller: the shape of almost every block



Six per cent of the cells, one hundred per cent of the behaviour.

A UART, a cache, a GPU. Different sizes, the same two halves.



The Worked Example: Accumulate N Samples

```
// the CONTROLLER decides. Narrow. No data passes through it.
always @(*) begin
    acc_clr = 0; acc_en = 0; cnt_ld = 0; cnt_dec = 0; done = 0;
    case (state)
        S_IDLE : if (start)    begin acc_clr=1; cnt_ld =1; end
        S_RUN  : if (!cnt_done) begin acc_en =1; cnt_dec=1; end
        S_DONE :               done    =1;
    endcase
end

// the DATAPATH holds and transforms. Wide. It decides nothing.
always @(posedge clk or negedge rst_n)
    if      (!rst_n)  sum <= 0;
    else if (acc_clr) sum <= 0;
    else if (acc_en)  sum <= sum + data;
```

Read the interface between them

Four control wires down, one status wire up. That is the entire contract. The controller never touches sum; the datapath never looks at state.



Measured, On the Accumulator

module	cells	flip-flops	what it contains
accum_ctrl	10	2	three states, five control outputs
accum_datapath	145	24	a 16-bit accumulator, an 8-bit down-counter
accum_top	156	26	both, plus the wires between them

Six per cent of the cells, one hundred per cent of the behaviour

That ratio is the whole argument for the split. You will re-time, widen and pipeline the expensive half many times over the life of a design — and each time, the half that decides what happens stays exactly as it was.

It also means the part that is hardest to get right is the part small enough to read in one sitting.

make dpctrl prints every control signal on every cycle, next to the accumulated sum.



The Three-Block Coding Pattern

How to write a state machine: the three-block pattern

BLOCK 1

state register

```
always @(posedge clk or negedge rst_n)
  if (!rst_n) state <= S_IDLE;
  else      state <= next_state;  sequential · <= · reset lives HERE
```

BLOCK 2

next-state logic

```
always @(*) begin
  next_state = state;  // the default
  case (state) ... endcase  end  combinational · = · no latch
```

BLOCK 3

output logic

```
always @(*) begin
  out = 1'b0;  // defaults
  case (state) ... endcase  end  state only => MOORE
```

Every state machine in this lab is written exactly like this. So is most industrial RTL.



Why Not One Block?

You can write a working state machine in a single clocked block, with the case statement inside it. Nothing about it is illegal.

	one block	three blocks
works?	yes	yes
where is the reset?	mixed in with the logic	block 1, and nowhere else
where is the clock?	wrapped around everything	block 1, and nowhere else
outputs are	registered, always	your choice — Moore or Mealy
reviewing a change	you re-read the timing every time	blocks 2 and 3 have no timing to re-read
adding a state	one place, easy to miss a branch	two places, both of them a case statement

One block is not wrong. Three blocks are easier to be right in, which over a project is the same thing.



The Default Assignment, and Why It Matters

```
always @(*) begin
    next_state = state;          // <-- THE DEFAULT ASSIGNMENT
    case (state)
        S_MAIN_GREEN  : if (car && timeout) next_state = S_MAIN_YELLOW;
        S_MAIN_YELLOW : if (timeout)       next_state = S_SIDE_GREEN;
        ...
    endcase
end

// Not one of those if statements has an else. Not one of them needs
// one: next_state was already written, unconditionally, on line 2.
// No path through this block leaves it unassigned, so no latch.
```

This is the idiom the lab linter had to learn

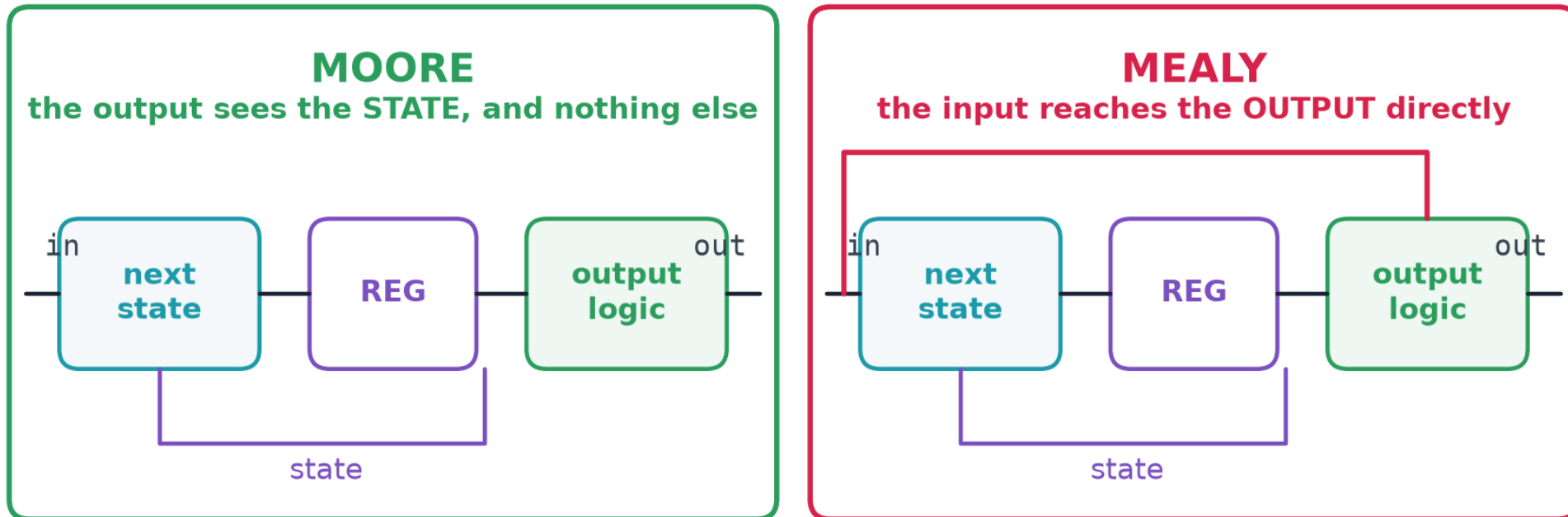
A naive latch rule flags every one of those if statements. Yosys builds a latch in none of them. make lintcheck puts the two opinions side by side on sixteen designs — and a linter that cries wolf gets switched off, which makes it worse than no linter at all.



Where the Output Comes From

Moore and Mealy: where the output comes from

One structural difference - whether the output logic can see the input. Everything else follows.



One structural difference — whether the output logic can see the input. Everything else follows.



The Differences, In One Table

Choosing between Moore and Mealy

	MOORE	MEALY
output depends on	state only	state AND input
output appears	one cycle after the input	the same cycle
output glitches	no - decoded from registers	yes - inherits the input's
states needed	usually one more	usually one fewer
measured here	13 cells, 2 flip-flops	14 cells, 2 flip-flops
use it when	the output leaves the block	you cannot afford the cycle

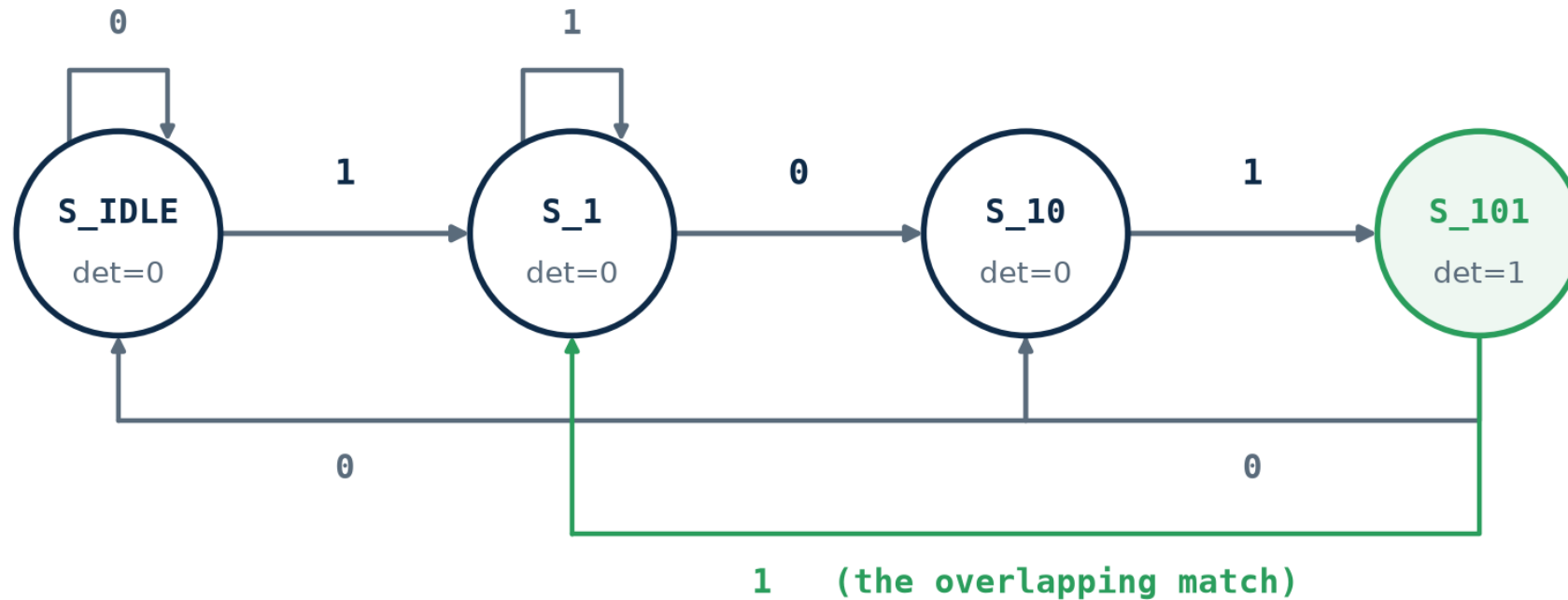
Both accept exactly the same language. They differ in WHEN they say so, not in WHAT they say.

Five rows. The last one is measured, not assumed.



The '101' Detector, Moore

The '101' detector as a MOORE state diagram



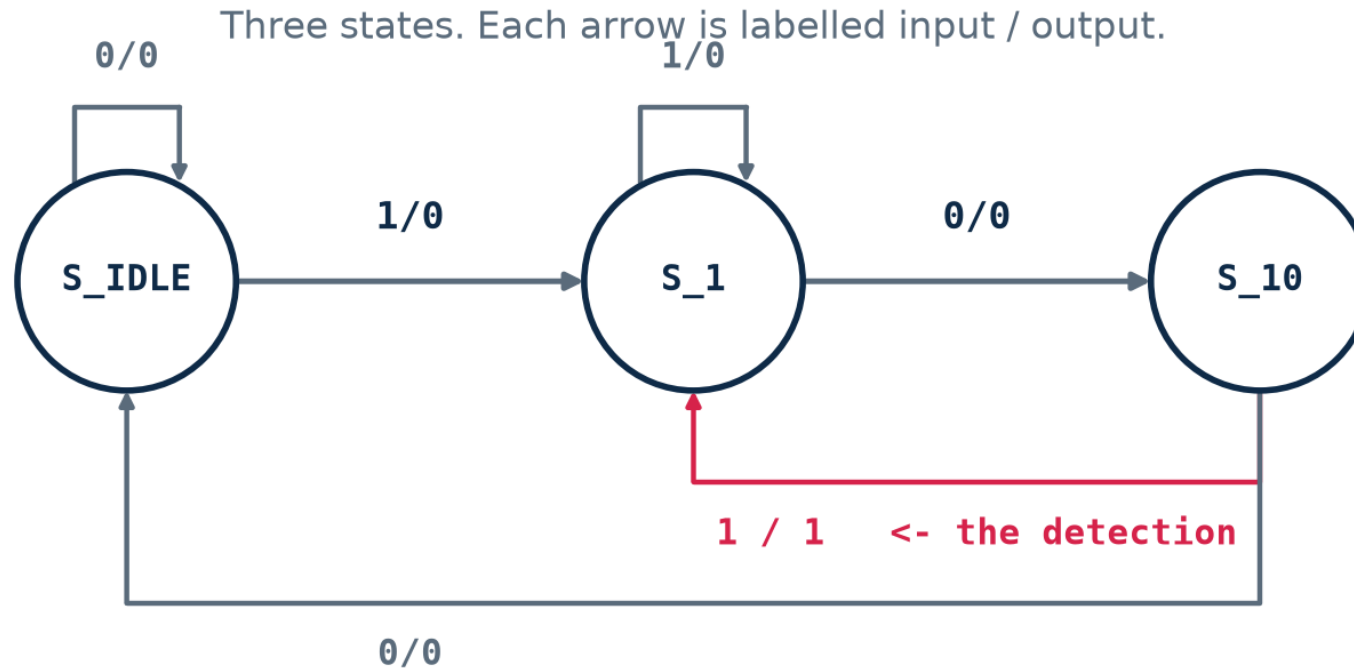
The fourth state exists only to say 'a match just finished'.

Four states. The output is decoded from the state register alone.



The Same Detector, Mealy

The same detector as a MEALY state diagram



The detection lives on an arrow, not in a state.

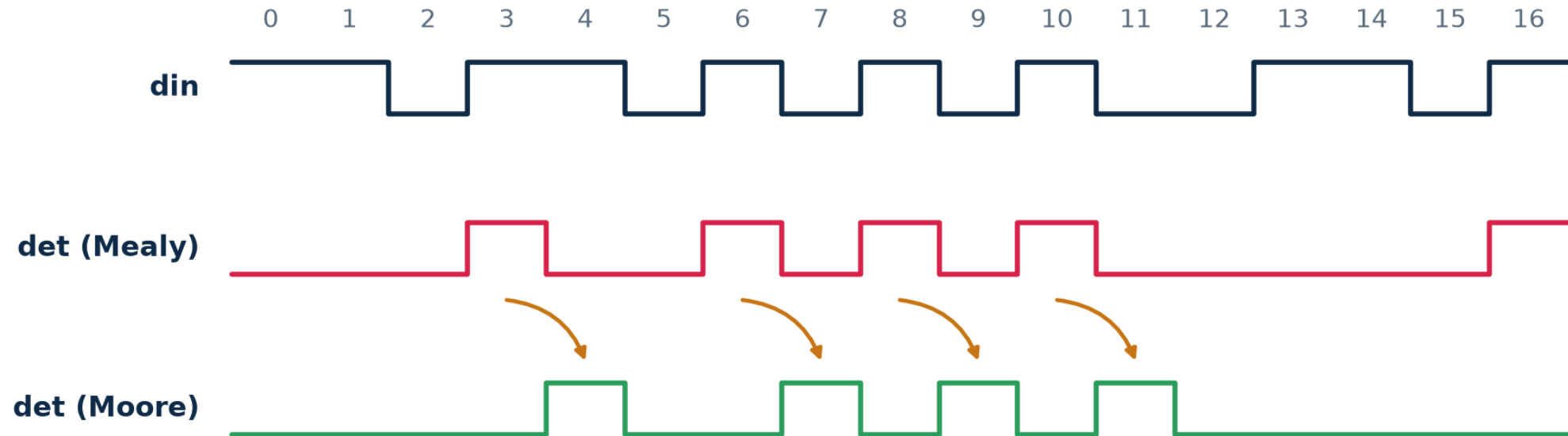
Three states. The output is written on the arrows, as input / output.



The One-Cycle Difference, Measured

The one-cycle difference, measured

One stream into both detectors, both checked against a golden model.



5 matches · 0 mismatches · Moore trails Mealy by exactly one cycle, every time

Five matches, zero mismatches against a golden model computed from the stream itself.



Choosing Between Them

CHOOSE MOORE

when the output leaves the block, drives other logic, or is timing critical. It comes out of a decode of registered bits, so it is clean, predictable and easy to constrain.

CHOOSE MEALY

when the cycle genuinely matters — a handshake that must be answered immediately, or a pipeline where one more cycle of latency costs throughput you cannot spare.

EITHER WAY, REGISTER IT AT THE BOUNDARY

A Mealy output that leaves your block hands the next designer a combinational path they did not ask for. If it crosses a module boundary, put a flip-flop on it — which turns it back into Moore.

The trap in the folklore

"Mealy needs fewer states" is true and almost never the reason to pick it. Here it saved one flip-flop's worth of nothing — both machines needed two — and cost one cell more. The real trade is the cycle against the glitch-free output, and that is a system question, not a coding one.



The Choice You Make By Typing Numbers

State encoding: the choice you make by typing numbers

state	binary	one-hot
S_IDLE	2'b00	4'b0001
S_1	2'b01	4'b0010
S_10	2'b10	4'b0100
S_101	2'b11	4'b1000

encoding	flip-flops	cells
binary	2	13
one-hot	4	30

measured by make fsm

On this generic gate library one-hot came out BIGGER, not smaller.

Four states need four flip-flops instead of two, and the next-state logic drives four bits instead of two.

Same behaviour, different hardware — and the direction of the difference is not the one usually quoted.



Which Encoding, and When

Which encoding, and when

encoding	the property	where it wins
binary	fewest flip-flops	ASIC, many states, area-critical
one-hot	one flip-flop per state; every decode is one wire	FPGA, speed-critical, few states
gray	one bit changes per legal transition	the state vector crosses a clock domain

In Verilog the encoding is yours. In VHDL it is the tool's, unless you say otherwise.

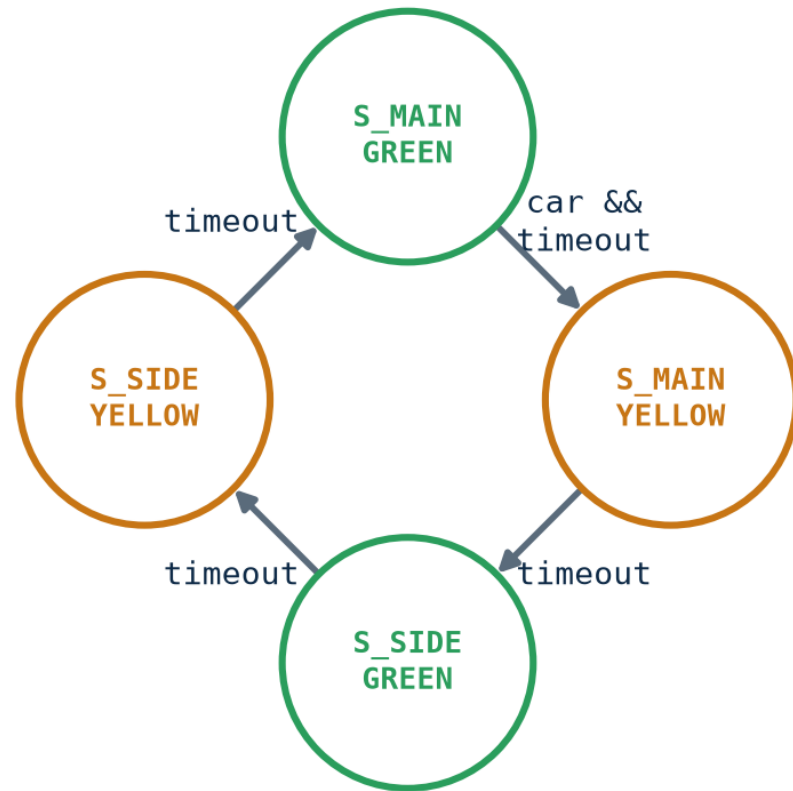
```
localparam S_IDLE = 2'd0      you wrote the bits
type state_t is (S_IDLE, ...) the tool picks the bits
```

Three encodings, three different reasons. None of them is a default.



The Traffic Light

A Moore controller with a datapath timer



state	MAIN	SIDE	cycles
S_MAIN_GREEN	GREEN	RED	6
S_MAIN_YELLOW	AMBER	RED	2
S_SIDE_GREEN	RED	GREEN	6
S_SIDE_YELLOW	RED	AMBER	2

40 cycles checked, 0 property violations

75 cells, 10 flip-flops - 2 for the state,
8 for the timer that counts out each phase

Two safety properties, checked on every one of forty cycles. Zero violations.



Checking a Property Instead of a Waveform

```
// P1 : the two roads are never green at the same time
if (main_light == GREEN && side_light == GREEN) begin
    $display("*** P1 VIOLATED at cycle %0d", cycles);
    errors = errors + 1;
end

// P2 : a green never goes straight to red - yellow comes first
if (prev_main == GREEN && main_light == RED) begin
    $display("*** P2 VIOLATED at cycle %0d", cycles);
    errors = errors + 1;
end
```

You write the rule once; the simulator tests it every cycle of every run

This is the cheap half of formal verification, available in any simulator, in about ten lines. It scales to runs far longer than anyone would page through by hand, and it keeps working after you change the design.

The expensive half — proving the property holds for every possible input — is what make prove does with a SAT solver.



Parameters, Hierarchy and Generate

From module to IP: parameters, hierarchy, generate

```
genvar k;  
generate  
  for (k=0; k<N; k=k+1) begin : stage  
    preg #(.W(W)) u_reg (  
      .d(tap[k]), .q(tap[k+1]));  
  end  
endgenerate
```

what you write

elaborate



what gets built (N = 4)

N	1	2	4	8
cells	8	16	32	64
flip-flops	8	16	32	64

Eight flip-flops per stage, exactly N stages, and no loop anywhere in the netlist.

Eight flip-flops per stage, exactly N stages, measured at four depths.



What Elaboration Actually Does

```
module delayline #(parameter W = 8, parameter N = 4)
    (input clk, rst, en, input [W-1:0] din,
     output [W-1:0] dout);

    wire [W-1:0] tap [0:N];    // one more element than stages
    assign tap[0] = din;

    genvar k;
    generate
        for (k = 0; k < N; k = k + 1) begin : stage
            preg #(.W(W)) u_reg (.clk(clk), .rst(rst), .en(en),
                                .d(tap[k]), .q(tap[k+1]));
        end
    endgenerate

    assign dout = tap[N];
endmodule
```

The for loop is an instruction to the ELABORATOR, not a loop in hardware. After elaboration there are N preg instances and no loop anywhere.



Eight Questions

#	Question	The answer in one line
1	What are the two halves of almost every block?	a datapath that holds, a controller that decides
2	What crosses between them?	control signals down, status signals up
3	Name the three blocks of an FSM.	state register, next-state logic, output logic
4	Why does a default assignment prevent a latch?	every output is written before any branch runs
5	What does the output logic read in a Moore machine?	the state, and nothing else
6	How much later does a Moore output appear?	exactly one clock cycle, every time
7	One-hot: fewer or more flip-flops than binary?	more — one per state instead of $\log_2(\text{states})$
8	Does a generate loop exist in the netlist?	no — it is elaborated away before synthesis

Theory 4 is about the notation — which turns out to be the least important part.

THEORY 4

Introduction to Hardware Description Languages

The syllabus says "HDLs such as Verilog or VHDL". The word doing the work is OR.

- Why an HDL is not a programming language
- Everything happens at once
- The parts of a module, and the word reg that confuses everyone
- Verilog and VHDL as reference cards, and a line-for-line mapping
- The anatomy of a testbench that checks itself
- Both languages actually run, and their transcripts diffed



An HDL Is Not a Programming Language

An HDL is not a programming language

It looks like one. That resemblance is the single biggest source of beginner bugs.

A PROGRAM

- statements run one after another
- a variable holds a value
- a loop repeats over time
- you describe a PROCEDURE

AN HDL DESCRIPTION

- every block runs ALL THE TIME
- a signal is a wire with a value on it
- a loop is unrolled into hardware
- you describe a STRUCTURE

The text is a blueprint, not a recipe - which is why "it compiles" means almost nothing.

You are not writing instructions for a machine to follow. You are writing a description of a machine.



Everything Happens At Once

Everything happens at once

The order you write the blocks in has no meaning whatsoever.

as written

```
assign p = a & b;
```

```
assign q = p | c;
```

```
assign r = q ^ d;
```

as built

AND

OR

XOR

*all three exist
at once, for ever*

You did not write a sequence. You wrote three facts about three pieces of hardware.

Write those three assign lines in any order you like — the circuit is identical.



The Parts of a Verilog Module

The parts of a Verilog module

```
module counter #(parameter WIDTH = 4) (  
    input          clk,  
    input          rst,  
    output reg [WIDTH-1:0] count  
);  
    always @(posedge clk) begin  
        if (rst) count <= 0;  
        else     count <= count + 1;  
    end  
endmodule
```

name, and a parameter for reuse

ports: the interface

reg = assigned in a block

clocked: flip-flops come from here

reg does NOT mean register.

What creates a flip-flop is assigning inside always @(posedge clk) - nothing else.

reg does not mean register. What creates a flip-flop is assigning inside always @(posedge clk) — nothing else.



How a Simulator Runs an HDL

How a simulator runs an HDL

- 1 A signal changes** at time t, some value is different
- 2 Wake everything sensitive to it** every always block and assign that reads it
- 3 Evaluate them all** in an order the standard does not fix
- 4 Schedule the results** non-blocking updates go into a queue
- 5 Apply the queue** all at once - this is what `<=` means
- 6 Repeat until nothing changes** then advance time

Step 3 is why RTL has coding rules at all: the evaluation order is genuinely unspecified.

The order blocks are evaluated in is genuinely unspecified. That is why non-blocking assignment exists.



Verilog On One Page

Verilog: the part that synthesises

Not the whole language - the part you need for this topic, which is much smaller.

THE MODULE

```
module counter4 (  
    input          clk,          a 1-bit input  
    input          rst_n,  
    output reg [3:0] count, assigned in a block  
    output          tc  
); ... endmodule          no semicolon after it
```

NETS AND VARIABLES

```
wire y;                                driven by assign  
reg q;                                assigned in always  
reg [7:0] bus;                        a vector, MSB first  
reg [7:0] mem [0:255];                an array of vectors  
localparam W = 8;                    constant, module scope  
parameter N = 4;                    the caller can set it
```

reg does NOT mean register. It means 'assigned in a procedural block'.

Not the whole language — the part that synthesises, which is much smaller.



Verilog: Blocks and Operators

Verilog: the three kinds of block, and the operators

THE THREE BLOCKS

<code>assign y = a & b;</code>	continuous
<code>always @(*) begin</code>	combinational
<code>y = a & b;</code>	blocking =
<code>end</code>	
<code>always @(posedge clk) begin</code>	sequential
<code>q <= d;</code>	non-blocking <=
<code>end</code>	

OPERATORS WORTH KNOWING

<code>& ^ ~</code>	bitwise
<code>&& !</code>	logical - one bit out
<code>== !=</code>	equality (x/z give x)
<code>=== !==</code>	identity - matches x, z
<code>{a, b}</code>	concatenation
<code>{4{a}}</code>	replication
<code>a[3:1]</code>	part select
<code>cond ? x : y</code>	a multiplexer

make subset measures where the synthesisable boundary actually is.

Three kinds of block, and the operators you will actually use.



VHDL On One Page

VHDL: the same ideas, spelled differently

Everything here has an exact counterpart on the Verilog page.

LIBRARIES AND ENTITY

```
library ieee;                                Verilog has none
use ieee.std_logic_1164.all;                  std_logic
use ieee.numeric_std.all; unsigned, arithmetic

entity counter is                            the INTERFACE only
    generic (WIDTH : positive);              = parameter
    port (clk : in std_logic);
end entity counter;
```

ARCHITECTURE

```
architecture rtl of counter is
    signal c : unsigned(3 downto 0);
begin                                        the IMPLEMENTATION
    process (clk)                            = always @(posedge)
    begin
        if rising_edge(clk) then
            c <= c + 1;                        <= assigns a signal
        end if; end process;
```

VHDL splits the interface from the implementation. Verilog puts both in one module.

Everything on this page has an exact counterpart on the previous one.



VHDL: Types, and What They Catch

VHDL: types, and what they catch

TYPES - THE REAL DIFFERENCE

<code>std_logic</code>	<code>0 1 X Z U W L H</code>
<code>std_logic_vector</code>	bits, no arithmetic
<code>unsigned / signed</code>	bits, with arithmetic
<code>integer</code>	constants and loops
<code>type state_t is</code>	a real enumerated
<code> (S_IDLE, S_1);</code>	type
Conversions are explicit,	and the compiler
every time.	will make you.

WHAT VHDL CATCHES

a non-exhaustive case	will not analyse
an illegal state value	not in the type
a width mismatch	8 into 4 is an error
signed mixed with unsigned	
The cost is keystrokes.	
The benefit is that the bug	
is found in the analyser,	
not in the lab.	

make langs – both languages, two simulators, transcripts diffed

The type system is the real difference between the two languages.



Verilog to VHDL, Line For Line (1 of 2)

Verilog to VHDL, line for line (1 of 2)

Verilog	VHDL	the difference that matters
module ... endmodule	entity + architecture	VHDL splits interface from body
input / output	port (... : in / out ...)	
parameter	generic	
localparam	constant	
wire	signal	VHDL has one kind, not two
reg	signal (or variable)	'reg' never meant register
[7:0]	(7 downto 0)	or (0 to 7) if you insist
generate for	for ... generate	

Learn one and you can read the other in an afternoon. These two tables are the afternoon.



Verilog to VHDL, Line For Line (2 of 2)

Verilog to VHDL, line for line (2 of 2)

Verilog	VHDL	the difference that matters
always @(posedge clk)	process (clk) + rising_edge(clk)	
always @(*)	process (all inputs)	process(all) in VHDL-2008
<= in a clocked block	<= signal assignment	same idea
= in a comb block	:= variable assignment	different symbol
case ... default	case ... when others	'when others' is compulsory
localparam S = 2'd0	type state_t is (S, ...)	a number against a type
\$display	report / write	

The syllabus says "HDLs such as Verilog or VHDL". The word doing the work is OR.

Nothing in this topic depends on which one you use.



What a Testbench Is Made Of

What a testbench is made of

1 CLOCK

```
always #5 clk = ~clk;
```

2 RESET

```
rst_n = 0; @(negedge clk); rst_n = 1;
```

3 STIMULUS

```
din = stream[i];
```

4 GOLDEN MODEL

```
exp[i] = s[i-2] & ~s[i-1] & s[i];
```

5 CHECKER

```
if (det !== exp[i]) errors = errors + 1;
```

6 VERDICT

```
if (errors==0) $display("PASS");
```

Step 4 is the one people leave out - and a testbench without it is not a test.

Topic 5 covers verification properly. These six parts are the minimum you need to finish Topic 2.



Verilog and VHDL, Side By Side

Verilog and VHDL: the same ideas, different notation

	Verilog	VHDL
origin	1984, Gateway Design Automation	1983, US Dept of Defense
standard	IEEE 1364, then 1800	IEEE 1076
feel	terse, C-like	verbose, Ada-like
typing	weak - it lets you do most things	strong - conversions are explicit
case sensitive	yes	no
interface / body	one module	entity and architecture
a clocked block	always @(posedge clk)	process (clk) + rising_edge
assignment	<= and =	<= and :=
used most in	North America, ASIC flows	Europe, defence, aerospace

None of those differences are about hardware.

Both describe registers, logic and hierarchy, and both synthesise to the same gates.

None of those differences are about hardware. Both describe registers, logic and hierarchy; both synthesise to the same gates.



Both Designs, In Both Languages, Actually Run

The same designs, in both languages, actually run

Verilog · iverilog

```
always @(posedge clk)
  if (rst) count <= 0;
  else if (en)
    count <= count + 1;
```

VHDL · ghdl

```
if rising_edge(clk) then
  if rst = '1' then
    cnt <= (others => '0');
  elsif en = '1' then
```

design	compared over	diff
counter	18 cycles	IDENTICAL
'101' detector	17 cycles	IDENTICAL

Not "they look similar" - the logs were compared line by line by diff.

Not "they look similar"

Both transcripts were compared line by line by diff, and there was nothing to report. The state machine is the interesting one: VHDL gives the states a real enumerated TYPE, so an illegal state will not compile. Verilog gives them numbers.



Which HDL Should You Learn?

Which HDL should you learn?

Learn the CONCEPTS. The notation follows in an afternoon.

registers and combinational logic · the clock edge · reset · hierarchy · the synthesisable subset

if you are doing	use	because
This course	Verilog	what Module 2 and the labs use
Most ASIC work	SystemVerilog	Verilog plus better types
European and defence work	VHDL	strong typing, long lifetimes
Increasingly	both	large projects mix them in one flow

The lab proves it rather than asserting it: two languages, two simulators, identical transcripts.

Learn the concepts. The notation follows in an afternoon.



Seven Questions

#	Question	The answer in one line
1	Name the biggest difference between an HDL and a program.	everything in an HDL runs all the time, concurrently
2	Does the order of assign statements matter?	no — they describe hardware that exists together
3	What does reg actually mean?	assigned in a procedural block; NOT a flip-flop
4	Why is the evaluation order unspecified?	the standard leaves it free; <= makes it not matter
5	Verilog or VHDL?	either — the concepts are identical, and the lab proves it
6	What does VHDL catch that Verilog does not?	illegal states, non-exhaustive cases, width mismatches
7	Where must a testbench's expected answer come from?	the stimulus — never from the design under test

PRACTICAL

Labs A-N

26 hours, 103 graded exercises, and not one number in this deck that you cannot reproduce yourself.

- Which tool answers which question, and how to install them
- Labs A-D: what RTL means, the ladder, proof, and the subset
- Labs E-H: the mismatch, the coding rules, style, and the two pitfalls
- Labs I-L: state machines, a controller with a timer, datapath and controller, and reuse
- Labs M-N: two languages, the whole flow, and the vendor tools



The Tools, and What Each One Is For

The tools, and what each one is for

tool	licence	what it does	used in
Icarus Verilog	free	simulate Verilog	this lab
GHDL	free	simulate VHDL	this lab
GTKWave	free	look at the waveforms	this lab
Yosys	free	synthesise, and prove equivalence	this lab
Verilator	free	fast simulation, a good linter	recommended
rtl_lint.py	you write it	the seven coding rules	this lab
Vivado	free WebPACK	synthesis for Xilinx parts	syllabus tool
ModelSim / Questa	free starter	simulation, both languages	syllabus tool
Design Compiler	commercial	ASIC synthesis	industry

Yosys is the interesting one: the only free tool here that both synthesises your RTL and PROVES the netlist it produced is equivalent to what you wrote.

Yosys is the interesting one: the only free tool here that both synthesises your RTL and PROVES the netlist matches it.



Two apt Lines, and the Whole Lab Runs

Install this, and the whole lab runs

REQUIRED

Debian / Ubuntu / WSL2

```
sudo apt update
```

```
sudo apt install yosys iverilog gtkwave python3
```

```
sudo apt install ghdl          # only for the VHDL comparison
```

```
yosys -V && iverilog -V && ghdl --version
```

Three version strings, and every target in the lab Makefile will run.

Without ghdl, everything except make langs still works.

No licence, no account, no download manager. Two lines and the whole lab runs.



The Vendor Tools the Syllabus Names

The vendor tools the syllabus names

	what to do	watch out for
1	Create a free AMD/Xilinx account	no account, no download
2	Download the Unified Installer (~200 MB)	not the 40 GB full archive
3	Choose Vivado ML Edition, then ML Standard	Standard is free; no licence file
4	Deselect every device family but yours	cuts 40 GB to about 8 GB
5	ModelSim: Intel FPGA Starter Edition is free	simulates both languages
6	Verify: vivado -version, vsim -version	both then run headless

They add a real device library, real timing numbers, and a flow you will meet at work.

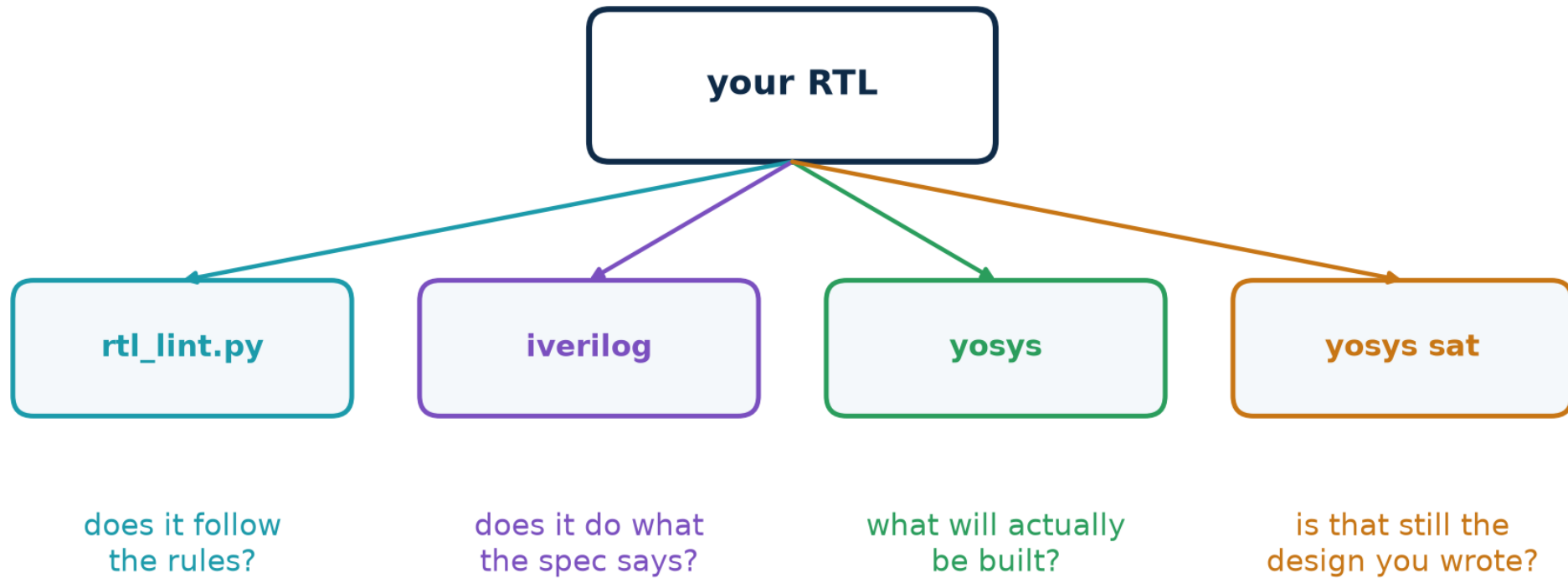
They add nothing to the CONCEPTS in this topic. Learn those on the free tools, where the flow takes seconds.

They add a real device library and a flow you will meet at work. They add nothing to the concepts.



Four Questions About One Piece of Code

Four questions about one piece of code



A methodology is the discipline of asking all four, every time.

Does it follow the rules? Does it do what the spec says? What will be built? Is that still your design?



Fourteen Parts, Twenty-Six Hours

The practical component - 26 hours, fourteen parts

A What RTL means 1 h	H The two pitfalls 2 h
B The abstraction ladder 2 h	I State machines 3 h
C Proof, not just testing 2 h	J A controller with a timer 2 h
D The synthesisable subset 2 h	K Datapath and controller 2 h
E Simulation against silicon 1 h	L From module to IP 2 h
F The coding rules 2 h	M Two languages 2 h
G Coding style 1 h	N The flow, and the vendor tools 2 h

103 graded exercises, every one with a worked solution

Every number in this deck came from a target in that lab.

A-D build the mental model, E-H the methodology, I-L the patterns every block is built from, M-N the connection to other languages and real tools.



What RTL Means, the Ladder, and Proof

A • what RTL means (1 h)

Run make transfer. Follow a single 5 through four registers.

Predict the value in each register on each cycle BEFORE running it.

Deliverable: the table, and why y is 1 on cycle 0.

B • the ladder (2 h)

Write the same full adder at all four levels.

Run make ladder — all four together, exhaustively.

Then synthesise each and compare cell counts.

Deliverable: why behavioural won.

C • proof (2 h)

Run make prove. Three pairs proved equivalent.

Then break the adder yourself and watch the checker catch it.

Deliverable: why 8 patterns was enough here and will not be next time.

The exercise that teaches the most is C

Delete a different term from fa_broken.v and predict which input pattern the solver will find. Getting that prediction right means you understand both the circuit and the tool.



The Subset, the Mismatch, and the Rules

```
$ make subset
s03_latch      OK      1      YES      inferred a LATCH: $_DLATCH_P_
s08_whileloop  REFUSED -      -      "only allowed in constant functions"
s10_divide     OK      371     no      a full combinational divider
s11_shift      OK      0      no      no logic at all - just wires

$ make mismatch
change b (list ASLEEP)  a=1 b=1  RTL y=0  NETLIST y=1  <-- THEY DISAGREE

$ make lintcheck
10 files · linter and Yosys agree on every one · 0 disagreements
```

Exercise	Task
D.1	Predict the cell count for each construct before running make subset
D.2	Add a twelfth construct of your own and predict the result
E.1	Explain the mismatch to someone who has not seen it, in three sentences
F.1	Add rule L008 to the linter — your choice — and justify it
F.2	Find a file the linter gets wrong, and say why regexes cannot fix it



Coding Style, and the Two Pitfalls

```
$ make mux
mux4_assign vs mux4_case      EQUIVALENT (proved, 92 SAT variables)
mux4_assign                   3 cells      0 flip-flops
mux4_if                        6 cells      0 flip-flops
mux4_case                     10 cells      0 flip-flops

$ make pitfalls
non-blocking version : 0 wrong cycles
blocking version      : 6 wrong cycles
shift_nb (non-blocking) 3 cells      3 flip-flops
shift_bl (blocking)     1 cells      1 flip-flops
s03_latch (missing else) 1 cells      *** 1 LATCH INFERRED ***
```

Exercise	Task
G.1	Predict the three cell counts before you run make mux
G.2	Write a fourth style of your own and predict where it lands
H.1	Predict q_bl for the first six cycles, then run it
H.2	Explain in two sentences why no tool warned about shift_bl
H.3	Add an else to s03_latch and confirm the latch disappears



State Machines, a Timer, and a Datapath

```
$ make fsm
matches in the stream : 5      mismatches vs golden : 0
PASS - same language, Moore trails Mealy by one cycle,
      and the one-hot encoding is indistinguishable
seq101_moore  binary          13 cells      2 flip-flops
seq101_moore  one-hot         30 cells      4 flip-flops
cycles checked : 40    property violations : 0      (traffic light)

$ make dpctrl
golden total : 157    hardware sum : 157
accum_ctrl   (controller)    10 cells      2 flip-flops
accum_datapath (datapath)    145 cells     24 flip-flops
```

Exercise	Task
I.1	Draw the state diagram for '110' before you write any code
I.2	Write it in both styles and confirm the one-cycle offset
I.3	Re-encode the Moore machine as gray and measure the cells
J.1	Add a pedestrian request input and a third property to check
K.1	Move the terminal-count test into the controller. What breaks, and why is it worse?



From a Module to an IP

```
$ make reuse
delayline mismatches : 0          prescaler ratio errors : 0

delayline #(W=8, N) synthesised at four different depths:
N = 1                8 cells      8 flip-flops
N = 2                16 cells     16 flip-flops
N = 4                32 cells     32 flip-flops
N = 8                64 cells     64 flip-flops
```

Exercise	Task
L.1	Predict the cell count for N = 16 before running it
L.2	Add a bypass parameter that removes the registers entirely when N = 0
L.3	Instantiate counter_n three times to divide by 4096, and check the ratio

L.2 is the one worth arguing about

A generate loop with N = 0 builds nothing, so dout must come straight from din. Getting that right means understanding that the parameter is resolved before any hardware exists — which is the whole point of the lab.



Two Languages, the Whole Flow, and the Vendor Tools

```
$ make langs
=== Verilog: iverilog ===      === VHDL: ghdl ===
IDENTICAL over all 18 cycles - including the wrap and the terminal count.
IDENTICAL over all 17 cycles, detections included.  (the '101' detector)

$ make flow
STAGE 2  LINT                0 issues
STAGE 4  SYNTHESIS           12 cells
STAGE 6  COMPARE             IDENTICAL on all 18 cycles
STAGE 7  PROVE               Equivalence PROVEN by induction

All seven stages passed.
```

Lab N: the same design in Vivado

Read the design, synthesise for a real part, report utilisation, write the netlist for gate-level simulation. Vivado is not installed in the environment these materials were built in, so lab N is the one whose output is not reproduced here — run it and record what you get.



The Same Flow in Vivado and ModelSim

The same flow in Vivado and ModelSim

1

`vlog counter.v ; vsim -c tb_counter`
ModelSim - the same job as iverilog + vvp

2

`read_verilog rtl/counter.v`
Vivado: read the design

3

`synth_design -top counter -part xc7a35t`
synthesise for a real device

4

`report_utilization`
LUTs and flip-flops - the same question as 'cells'

5

`write_verilog -mode funcsim net.v`
the netlist, for gate-level simulation

Stated plainly: these commands were NOT run here. Every number in this deck came from the free toolchain.

Stated plainly on the slide: these commands were not run here. Every number in this deck came from the free toolchain.



How the 103 Exercises Are Weighted

How the 103 exercises are weighted

part	exercises	weight	part	exercises	weight
A · what RTL means	6	5%	H · the two pitfalls	7	7%
B · the ladder	9	8%	I · state machines	12	13%
C · proof	7	7%	J · controller + timer	6	6%
D · the subset	10	9%	K · datapath/controller	7	8%
E · sim vs silicon	5	5%	L · module to IP	6	6%
F · coding rules	10	9%	M · two languages	5	5%
G · coding style	5	5%	N · the flow	8	7%

Predict, then measure.

An exercise where you ran the command and copied the answer earns very little; one where you predicted first earns it all.

Predict, then measure. Being wrong and knowing why is the whole point of a lab.



What This Lab Does Not Show You

- `rtl_lint.py` is regular expressions, not a Verilog parser. It reads code the way a careful reviewer skims it and can be fooled by unusual formatting.
- The cell counts come from Yosys' generic library, not a foundry library. They compare designs against each other; they are not area figures.
- `fa_switch.v` is a functional transistor model. There are no threshold voltages and no capacitances — SPICE is the tool for that.
- Yosys accepts initial blocks because it targets FPGAs. An ASIC flow would not, and that difference is a genuine trap.
- Vivado and ModelSim were not run. Every measured number here came from iverilog, ghdl and yosys, and is reproducible with make.
- Nothing here covers verification methodology beyond a directed testbench and a handful of assertions — that is Topic 5, a large subject on its own.
- The state machines here are small and single-clock. Clock-domain crossing, reset synchronisers and safe-state recovery are all real, and all later.



Topic 2 In Fourteen Lines

- RTL says which registers exist and what transfers into them. Nothing else.
- There are two kinds of logic, combinational and sequential, and no third.
- `<=` reads every right-hand side first, then updates together. `=` does not — and the blocking version built 1 flip-flop where 3 were meant.
- Four levels of abstraction describe the same circuit — proved, not assumed.
- Write at the highest level that says what you mean: behavioural gave 5 cells against 6.
- Not all legal Verilog is synthesisable, and `a / b` cost 371 cells against 0.
- A combinational block that does not assign on every path builds a latch; a default assignment at the top of the block prevents it.
- An incomplete sensitivity list makes simulation and silicon disagree silently.
- Equivalent is not identical: three mux styles, 3, 6 and 10 cells.
- Almost every block is a datapath and a controller. Here: 145 cells against 10.
- Write state machines in three blocks — register, next state, output.
- Moore trails Mealy by exactly one cycle, and one-hot was BIGGER here.
- `parameter`, `hierarchy` and `generate` are elaborated away before synthesis.
- Verilog and VHDL produced identical transcripts, on both designs.



CLOSE

What To Do Next

By the end of this subtopic you can

- ✓ Read an always block and say which registers it creates.
- ✓ Tell combinational logic from sequential, and code each correctly.
- ✓ Write the same function at four levels of abstraction, and say what each level costs you.
- ✓ Prove two descriptions are the same circuit, instead of hoping.
- ✓ Say which Verilog constructs synthesise, which do not, and which are traps.
- ✓ Recognise an inferred latch before the tool builds one.
- ✓ Explain why an incomplete sensitivity list is worse than an error.
- ✓ Apply the seven coding rules, and say why each one exists.
- ✓ Split a block into a datapath and a controller.
- ✓ Write a state machine in the three-block form, in either style.
- ✓ Parameterise a design so one file covers a family of widths.
- ✓ Read a VHDL design without needing it translated.
- ✓ Take a design from spec to formal proof, showing evidence at each stage.

And one habit: before you write RTL, know what you expect the tool to build. Then check whether it did.

Coming next in Module 2

Topic 3 covers the digital logic this all rests on. Topic 4 writes much more RTL. Topic 5 is verification — how you know it works. Topic 6 is timing — whether it is fast enough.